DEEP LEARNING AI ASSIGNMENT

In Collaboration with

VECTORIUM LABS

Multivariate Time-Series Prediction Using Deep Learning

(Appliance Energy Prediction Using the Energy Dataset)

Kavindu Induwara Jayasinghe

13/04/2026

Table of Contents

1. Introduction 5

2. Data Insights 6

3. Feature Engineering 11

3.1 Feature Selection 11

3.2 Outlier Treatment 11

3.3 Time-Based Features 12

3.4 Lag Features 12

3.5 Rolling Features 13

3.6 Interaction Features 14

3.7 Summary 14

4. Data Preprocessing 15

4.1 Missing Values 15

4.2 Train Test Split 15

4.3 Data Scaling 15

5. Model Development 16

5.1 Baseline Models 16

5.1.1 Linear Regression 16

5.1.2 Random Forest 17

5.2 Deep Learning Model 18

5.2.1 LSTM (Long-Short Term Memory) 18

5.2.2 Architecture 18

5.3 Evaluation & Comparison 19

6. Model Optimization 21

6.1 Random Forest Optimization 21

6.2 LSTM Optimization 22

7. Challenges and Solutions 24

7.1 Data Preparation and Feature Engineering Challenges 24

7.2 Model Selection and Performance Challenge 24

8. Conclusion 25

8.1 Key Findings 25

8.2 Limitations 25
8.3 Future Enhancements 26
9. References 27
10. Appendix 27

Table of Figures

Figure 1: Statistical Summary & Missing Value Check 6
Figure 2: Appliance Over Time 6
Figure 3: Average Energy Consumption by Hour 7
Figure 4: Correlation Heatmap 8
Figure 5: Boxplot of Core Features 9
Figure 6: Frequency Distribution of Appliances 9
Figure 7: Autocorrelation Plot 10
Figure 8: Time-based Feature Generation 12
Figure 9: lagged Features 13
Figure 10: Rolling Features 13
Figure 11: Example of Interaction Features 14
Figure 12: Linear Regression Actual Vs Predicted 17
Figure 13: Random Forest Actual Vs Predicted 17
Figure 14: LSTM Architecture 18
Figure 15: LSTM Training Setup 19
Figure 16: LSTM Actual Vs Predicted 19
Figure 17: Optimized Random Forest Actual vs Predicted 21
Figure 18: Baseline LSTM Training and Validation Loss 23
Figure 19: Optimized LSTM Training and Validation Loss 23

Tables

Table 1: Model Comparison 20
Table 2: Random Forest Baseline vs Optimized 22
Table 3: LSTM Baseline vs Optimized 23

1. INTRODUCTION

This project assignment focuses on appliance energy consumption prediction leveraging a multivariate time-series dataset. The dataset consists of several operational, environmental, and temporal factors recorded at regular intervals of 10 minutes, making it subtle for a time-series analysis.

When energy efficiency is considered along with cost reduction and resource management in residential and commercial environments, understanding and forecasting appliance energy consumption become significantly critical

The primary objective of this assessment is to design a predictive system which can estimate appliance energy consumption accurately based on historical trends and other influencing attributes.

- To achieve this, a structured approach is followed:
- Analyzing relationships and data patterns through an exploratory data analysis
- Engineering more meaningful features to improve forecasting capability
- Building both conventional machine learning and deep learning models
- Evaluating models and optimizing the model's performance

Both the traditional models and sequence based deep learning models are experimented to get a better understanding regarding which approach possesses better performance, especially for this type of specific structured time-series data.

2. DATA INSIGHTS

An Exploratory Data Analysis (EDA) was performed to identify patterns, understand structure of the dataset, analyze relationships between attributes, and detect anomalies.

First, a basic inspection of the dataset is done through exploring the statistical summary of the numerical columns and checking for missing values within the raw dataset.

```
# Display statistical summary for numeric columns
energy_df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
Appliances	19735.0	97.694958	102.524891	10.000000	50.000000	60.000000	100.000000	1080.000000
lights	19735.0	3.801875	7.935988	0.000000	0.000000	0.000000	0.000000	70.000000
T1	19735.0	21.686571	1.606066	16.790000	20.760000	21.600000	22.600000	26.260000
RH_1	19735.0	40.259739	3.979299	27.023333	37.333333	39.656667	43.066667	63.360000
T2	19735.0	20.341219	2.192974	16.100000	18.790000	20.000000	21.500000	29.856667
RH_2	19735.0	40.420420	4.069813	20.463333	37.900000	40.500000	43.260000	56.026667
T3	19735.0	22.267611	2.006111	17.200000	20.790000	22.100000	23.290000	29.236000
RH_3	19735.0	39.242500	3.254576	28.766667	36.900000	38.530000	41.760000	50.163333
T4	19735.0	20.855335	2.042884	15.100000	19.530000	20.666667	22.100000	26.200000
RH_4	19735.0	39.026904	4.341321	27.660000	35.530000	38.400000	42.156667	51.090000
T5	19735.0	19.592106	1.844623	15.330000	18.277500	19.390000	20.619643	25.795000
RH_5	19735.0	50.949283	9.022034	29.815000	45.400000	49.090000	53.663333	96.321667
T6	19735.0	7.910939	6.090347	-6.065000	3.626667	7.300000	11.256000	28.290000
RH_6	19735.0	54.609083	31.149806	1.000000	30.025000	55.290000	83.226667	99.900000
T7	19735.0	20.267106	2.109993	15.390000	18.700000	20.033333	21.600000	26.000000
RH_7	19735.0	35.388200	5.114208	23.200000	31.500000	34.863333	39.000000	51.400000
T8	19735.0	22.029107	1.956162	16.306667	20.790000	22.100000	23.390000	27.230000
RH_8	19735.0	42.936165	5.224361	29.600000	39.066667	42.375000	46.536000	58.780000
T9	19735.0	19.485828	2.014712	14.890000	18.000000	19.390000	20.600000	24.500000
RH_9	19735.0	41.552401	4.151497	29.166667	38.500000	40.900000	44.338095	53.326667
T_out	19735.0	7.411665	5.317409	-5.000000	3.666667	6.916667	10.408333	26.100000

```
# Check missing values
missing_summary_df = summarize_missing_values(energy_df)
missing_summary_df
```

	missing_count	missing_percentage
Appliances	0	0.0
lights	0	0.0
T1	0	0.0
RH_1	0	0.0
T2	0	0.0
RH_2	0	0.0
T3	0	0.0
RH_3	0	0.0
T4	0	0.0
RH_4	0	0.0
T5	0	0.0
RH_5	0	0.0
T6	0	0.0
RH_6	0	0.0
T7	0	0.0
RH_7	0	0.0
T8	0	0.0
RH_8	0	0.0

Figure 1: Statistical Summary & Missing Value Check

The time-series visualization of appliance energy consumption illustrates distinctive fluctuations over time when appliances are plotted over time. Appliance energy usage is not constant and changes depending on various time-dependent features such as daily routines.

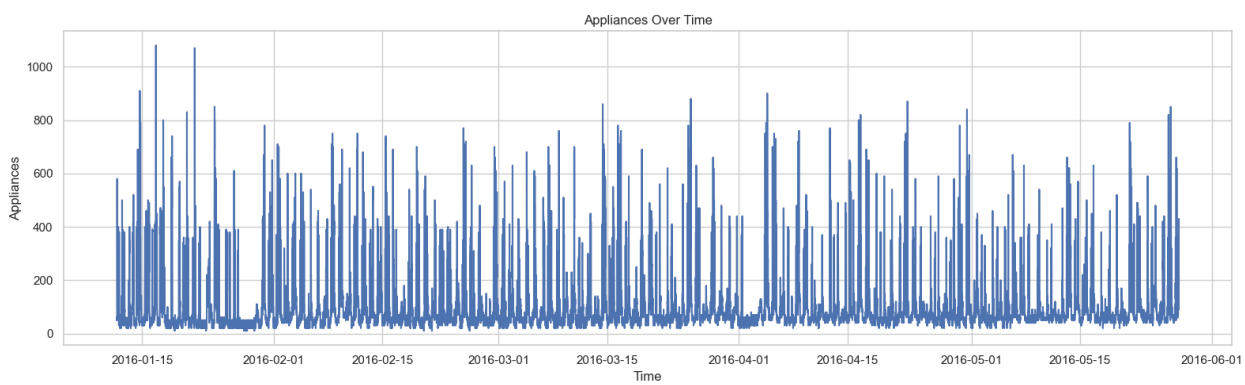

Figure 2: Appliance Over Time

From this plot, it is revealed that,

- Appliance energy consumption follows periodic patterns
- There are significantly visible spikes and drops
- Peaks indicate sudden high energy usage time periods

To understand the daily patterns at a much deeper level, the average energy consumption was explored across hours of the day.

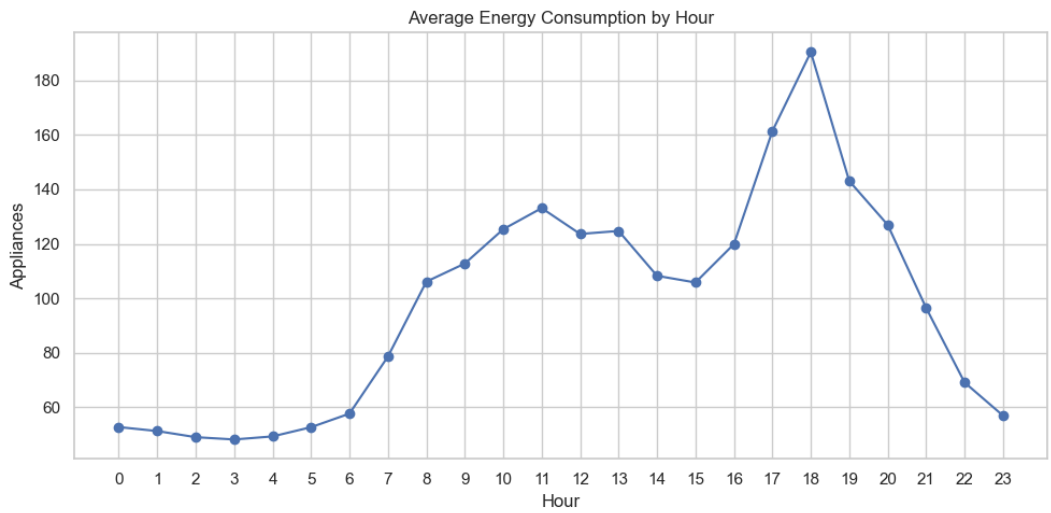

Figure 3: Average Energy Consumption by Hour

This plot shows that energy consumption significantly increases during active hours and during evening hours are when the energy consumption peaks. During late night and early morning hours a lower energy consumption is observed.

Additionally, a correlation heatmap was generated to better understand the correlations between attributes.

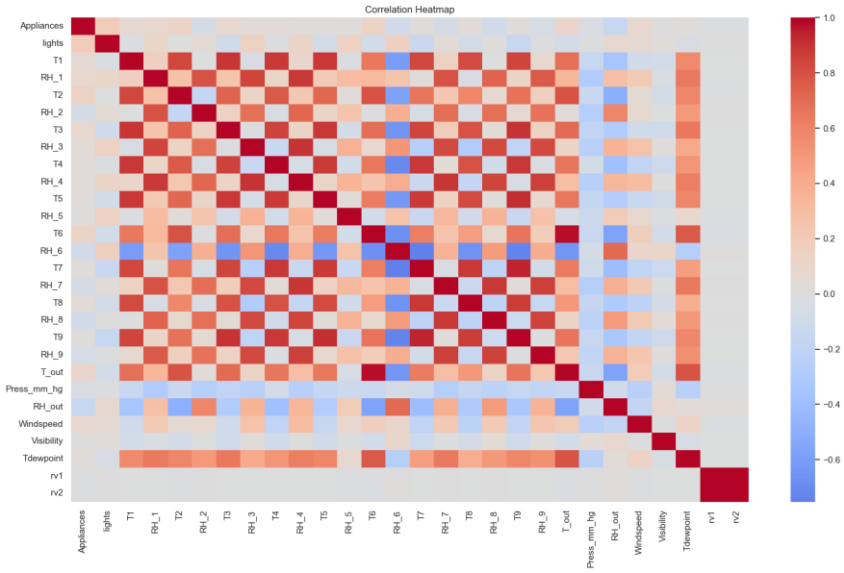

Figure 4: Correlation Heatmap

Key Observations:

Indoor temperature related attributes (T1 – T9) are highly correlated with each other, therefore instead of using all the temperature features, only T1 was selected later as it represents all the indoor temperature attributes. Similarly, humidity-related features (RH_1 – RH_9) are also strongly correlated, which limits the selection of one humidity attribute to represents all humidity-related attributes. This multicore linearity highlights the redundancy among these clusters of features. However, other variables such as outdoor conditions and lights show moderate or low relationships with the target attribute.

These insights from the correlation-heatmap aided the feature selection process through helping identify which features carry independent information and which are redundant. Because of this, the feature selection process was done easily avoiding complicated approaches like PCA (Principal Component Analysis).

With the selected features, boxplots were visualized to detect the presence of outliers in core features.

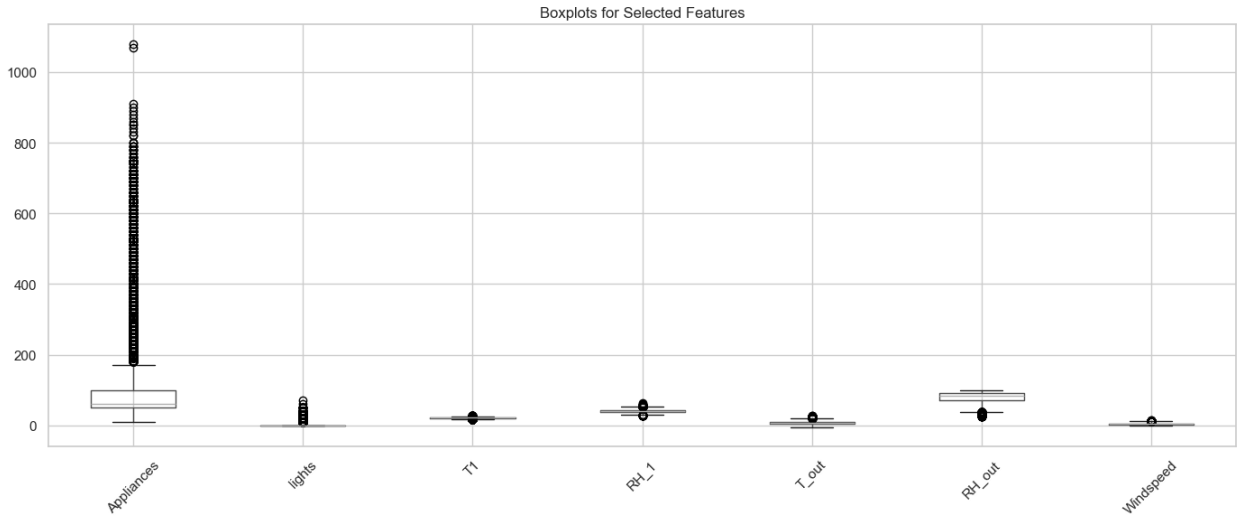

Figure 5: Boxplot of Core Features

The boxplot visualization revealed extreme deviations in appliance energy along with indicating minor outliers in some environmental features as well. These outliers may represent real-world variations, and simply not just errors.

With the purpose of understanding the spread of the target attribute, the distribution of appliance energy consumption was explored and analyzed. The resulting histogram indicated that the data is right-skewed, meaning most energy usage values are relatively low, with minor instances of very high energy consumption. The presence of high-usage periods is depicted through this uneven distribution.

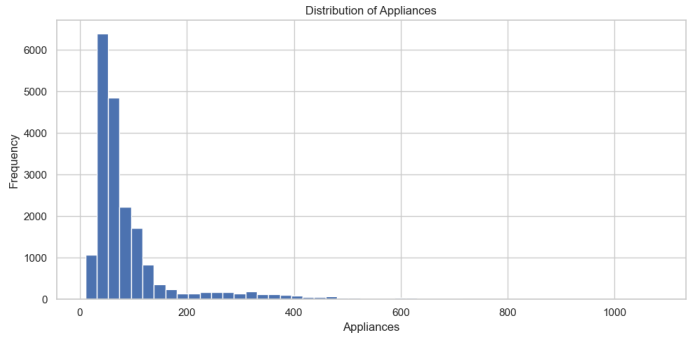

Figure 6: Frequency Distribution of Appliances

Overall, the EDA showcases multiple important characteristics of the selected dataset. Appliance energy consumption is significantly affected by time and human activity, while several environmental factors exhibit high redundancy caused by strong correlations. These insights are critical as they inform the processes of feature selection and preprocessing stages that follow after.

Next, autocorrelation analysis was performed using *acf plot* from "*statsmodel*" in order to decide the optimal lag values.

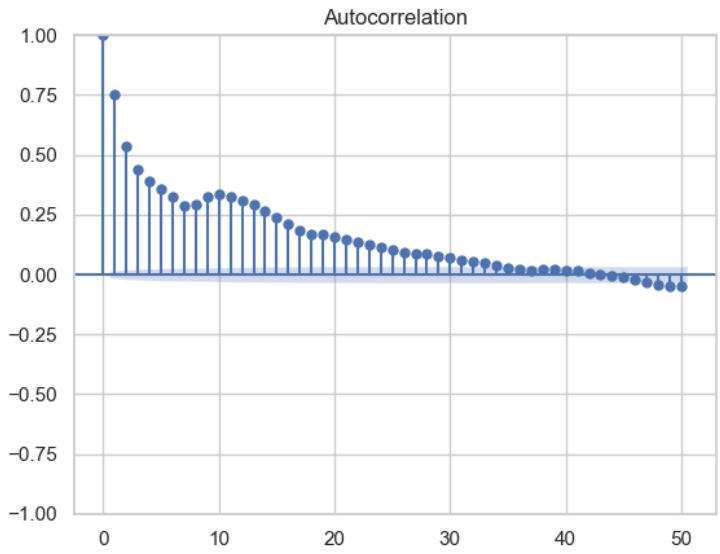

Figure 7: Autocorrelation Plot

The autocorrelation plot demonstrates strong positive correlation particularly at lower lags, highlighting recent past values significantly influence current appliance rather than old values. The gradual decay of correlation over increasing lags reflects the presence of both short-term and moderate long-term dependencies.

3. FEATURE ENGINEERING

Feature Engineering procedure was conducted with the purpose of improving the forecasting capability of the dataset by generating new features or aspects of features which better extract relationships, dependencies, and temporal patterns influencing the appliance energy consumption. Since the dataset itself is time-series in nature, significant focus was given to engineering features which reflect both historical dependencies and time-based behavior.

3.1 Feature Selection

Feature selection was done to minimize redundancy and enhance model efficiency through retaining only the most suited variables. Based on the correlation analysis, several temperature and humidity attributes showcased high inter-correlation, reflecting that they provide similar information. Therefore, representative features like T1 and RH_1 were chosen instead of including all related attributes. Additionally, features like lights, T_out, RH_out, and Windspeed were retained since they provide independent and meaningful information related to appliance energy consumption. This methodical approach helps minimize multicollinearity, model simplification, and enhance generalization while preserving critical information.

3.2 Outlier Treatment

To manage extreme or deviated values in the dataset, outlier treatment was performed. Leveraging the insights illustrated from the boxplots generated during the EDA, selected attributes like appliance energy consumption, lighting usage, and other environmental were examined for outliers. Instead of excluding these observations, which could surely disrupt the temporal continuity of the time-series data, an Interquartile Range (IQR)-based clipping strategy was applied. This strategy clips or limits deviated values and keeps them within a defined range while preserving the dataset structure, guaranteeing high-consumption events are not completely ignored. In this case, standard statistical 1.5 was used as the IQR multiplier. So, it helps to control the extreme values by defining upper and lower boundaries as follows:

$$Lower\ limit = Q1 - IQR_{multiplier} \times IQR_{value}$$

$$Upper\ limit = Q3 - IQR_{multiplier} \times IQR_{value}$$

Where, $IQR_{multiplier} = 1.5$ and $IQR_{value} = Q3 - Q1$

3.3 Time-Based Features

One of the primary engineering assignments was the generation of time-based features derived from the date column (Note: date column was converted into datetime format prior to these feature engineering). Attributes like hour of the day, day of the week, month, and an indicator of weekends were generated. These specific temporal variables aid the model to understand periodic patterns associated with human activity, as energy consumption basically changes depending on daily routines, weekly schedules, or seasonal patterns. For instance, higher energy consumption is observed during most evenings and as well as weekends when occupancy and appliance usage are higher.

2) Creating time-based features

```python
energy_df = create_time_features(energy_df)

energy_df[
    ["Appliances", "hour", "day_of_week", "month", "day_of_month", "is_weekend"]
].head()
```

[8] Python

date	Appliances	hour	day_of_week	month	day_of_month	is_weekend
2016-01-11 17:00:00	60	17	0	1	11	0
2016-01-11 17:10:00	60	17	0	1	11	0
2016-01-11 17:20:00	50	17	0	1	11	0
2016-01-11 17:30:00	50	17	0	1	11	0
2016-01-11 17:40:00	60	17	0	1	11	0

- Time-related variables were created to capture temporal usage patterns in appliance energy consumption.
- These include hour of the day, day of the week, month, day of the month, and a weekend indicator.

Figure 8: Time-based Feature Generation

3.4 Lag Features

In addition to the above-mentioned time-based features, feature engineering stage also took part in creating lag-based features in order to extract the sequential dependency present in the time-series data. These lag features indicate past appliance energy consumption values at several time intervals, such as 10mins, 30mins, and one hour prior to a specific instance. These values were decided based on the autocorrelation analysis through identifying points where the correlation was relatively high and gradually decaying. Instead choosing all lags or choosing the first most correlated lags, these specific values were intentionally selected to capture short-term and medium-

term temporal dependencies while avoiding overfitting and redundancy. By integrating these lagged values, the model can have a better understanding om how previous energy usage impacted future consumption, which is especially crucial for forecasting tasks.

	Appliances	Appliances_lag_1	Appliances_lag_3	Appliances_lag_6
date				
2016-01-11 17:00:00	60	NaN	NaN	NaN
2016-01-11 17:10:00	60	60.0	NaN	NaN
2016-01-11 17:20:00	50	60.0	NaN	NaN
2016-01-11 17:30:00	50	50.0	60.0	NaN
2016-01-11 17:40:00	60	50.0	60.0	NaN
2016-01-11 17:50:00	50	60.0	50.0	NaN
2016-01-11 18:00:00	60	50.0	50.0	60.0
2016-01-11 18:10:00	60	60.0	60.0	60.0
2016-01-11 18:20:00	60	60.0	50.0	50.0
2016-01-11 18:30:00	70	60.0	60.0	50.0

Figure 9: lagged Features

3.5 Rolling Features

Furthermore, with the idea of capturing short-term trends, rolling window features were taken into account. To smooth out the rigid fluctuations and identify underlying data patterns, moving average over specified window sizes of 1hour and 2 hours were calculated. These rolling average values furnish the model with underlying information on recent trends instead of solely relying on individual historical observations, enhancing the capability to handle variability and noise.

	Appliances	Appliances_rolling_mean_6	Appliances_rolling_mean_12
date			
2016-01-11 17:00:00	60	NaN	NaN
2016-01-11 17:10:00	60	NaN	NaN
2016-01-11 17:20:00	50	NaN	NaN
2016-01-11 17:30:00	50	NaN	NaN
2016-01-11 17:40:00	60	NaN	NaN
2016-01-11 17:50:00	50	55.000000	NaN
2016-01-11 18:00:00	60	55.000000	NaN
2016-01-11 18:10:00	60	55.000000	NaN
2016-01-11 18:20:00	60	56.666667	NaN
2016-01-11 18:30:00	70	60.000000	NaN
2016-01-11 18:40:00	230	88.333333	NaN
2016-01-11 18:50:00	580	176.666667	115.833333
2016-01-11 19:00:00	430	238.333333	146.666667
2016-01-11 19:10:00	250	270.000000	162.500000
2016-01-11 19:20:00	100	276.666667	166.666667

Figure 10: Rolling Features

3.6 Interaction Features

The generation of interaction features was included with the purpose of capturing combined effects between domain specific environmental variables. This is significant as the impact of one variable often depends on the level of another. For instance, a high temperature alone may not always significantly increase energy usage, but when combined with high humidity, it may lead to increased usage of fans or air conditioners. Similarly, moderate temperature with low humidity might result in relatively lower appliance usage compared to the same temperature under humid conditions.

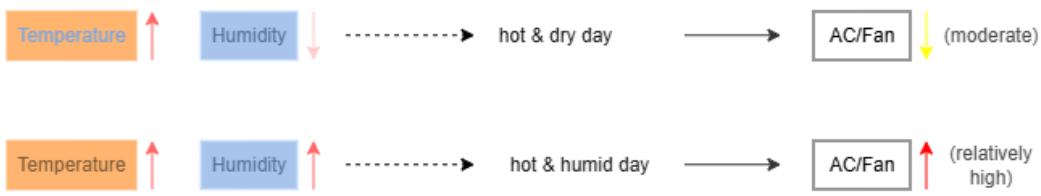

Figure 11: Example of Interaction Features

The multiplication logic was intentionally selected instead of other operations to capture the idea of interaction strength, where the combined effect reflects a stronger impact when both variables are high. In contrast, an operation like addition would have treated the attributes independently, simply summing the values avoiding the emphasizing joined effect. With the use of multiplication logic, the model possesses the ability to capture non-linear relationships especially when the influence of one attribute depends on the presence of another. This made the feature more expressive and better suited for this specific domain of energy usage.

3.7 Summary

One important aspect of feature engineering in this project that is mentioned is the handling of feature redundancy. As observed in the correlation heatmap during the EDA, multiple temperature and humidity indicators showcased very high correlations with one another. Instead of selecting all such features, only representatives were included to avoid redundancy while preserving overall information content. This method contributes to reducing multicollinearity, simplifying the mode and improving generalization.

Altogether, the feature engineering procedure was able to transform the original dataset into a more informative and richer representation, allowing the models to better understand both environmental influences and temporal dynamics on appliance energy consumption.

4. DATA PREPROCESSING

Data preprocessing procedures were performed after the feature engineering stage to prepare the dataset for model training stage. This stage makes sure that the dataset is clean, consistent, and best suited for machine leaning models as well as deep learning models. Since the feature engineering strategies introduced additional features like lag and rolling attributes, preprocessing was necessary to handle missing values and standardized the dataset.

4.1 Missing Values

One of the initial steps in preprocessing was the missing values handling part. The original dataset did not include significant missing values. However, as a result generating of lagged and rolling features, null values were spawned at the beginning of the dataset. As a result, even though handling missing values is a step that comes under the preprocessing stage, this was addressed at the end of feature engineering stage itself. During that stage the affected rows were removed to preserve data integrity. Therefore, there was no need to additionally handle missing values during the preprocessing stage. This strategy is best suited for time-series data, since the number of dropped rows is significantly small compared to the overall dataset and does not affect the overall learning process.

4.2 Train Test Split

Then, a traditional chronological train-test split was done to divide the dataset into subsets of training and testing. This time-series data was divided based on time, using the initial 80% of the observations for training and the remaining data of 20% for testing. This method preserves the temporal order of the time-series dataset and avoids data leakage from future observations.

4.3 Data Scaling

After the dataset split, to normalize the range of input features, scaling was applied. The dataset contains attributes with different units and scales like temperature, humidity, and appliance energy consumption. Therefore, scaling was required to make sure that no individual feature will dominate

the learning process. Using the "*StandardScaler*" method from "*sklearn*" library, the features were transformed using standardization so that they all possess a mean of zero and a standard deviation of one. The scaler was fitted to the training and testing subsets independently, ensuring that no information from the test set affected the training process. StandardScaler was selected instead of MinMaxScaler due to the existence of outliers and the skewed distribution in the dataset, especially in the target variable as MinMax is highly sensitive to outliers.

5. MODEL DEVELOPMENT

Under model development phase, several approaches for forecasting appliance energy consumption are considered for evaluation with the purpose of identifying the most appropriate method for this sort of structured time-series data. Both conventional machine learning models (Linear Regression, Random Forest) and a deep learning model (LSTM) were developed to enable a comprehensive comparison between simpler and more complex modelling strategies. Additionally, this phase also includes training models on the processed dataset and evaluating and comparing their performance using relevant evaluation metrics.

5.1 Baseline Models

5.1.1 Linear Regression

The initial step included implementing baseline models to establish reference performance levels. As the simplest model, Linear Regression was selected as it aligns a linear relationship between the input features and the target variables of the dataset. This model tries to predict appliance energy usage as a weighted combination of the input features.

The purpose of this Regression approach is not only to create predictions but also to act as a benchmark against which more complex machine learning and deep learning models can be evaluated. Linear Regression model helps in understanding whether the selected dataset can be modeled effectively using basic linear relationships due to its simplicity and interpretability.

During training phase, the regression model learns coefficients for each feature by minimizing the loss in the training data. Then the trained model was used to get predictions on the test dataset, allowing evaluation of its own generalization performance.

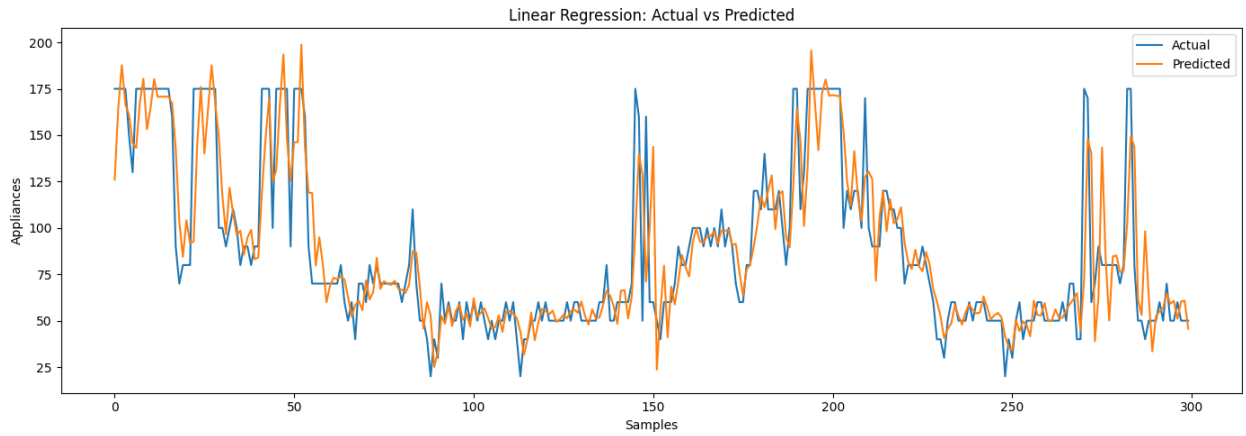

Figure 12: Linear Regression Actual Vs Predicted

5.1.2 Random Forest

In addition to the linear regression model, a Random Forest Regressor was developed as a more advanced yet baseline approach. Random Forest is an ensemble learning strategy that builds multiple decision trees then combines their outcomes to enhance predictive performance. It is especially effective in capturing non-linear relationships and interactions within features, reflecting that it is well-suited for a structured dataset consisting of engineered features.

The model was trained using the processed and scaled feature set, where each decision tree is responsible for the final prediction through an averaging procedure. The ensemble method helps to minimize overfitting and enhances robustness. Predictions which are generated on the test dataset were used to evaluate the model's effectiveness.

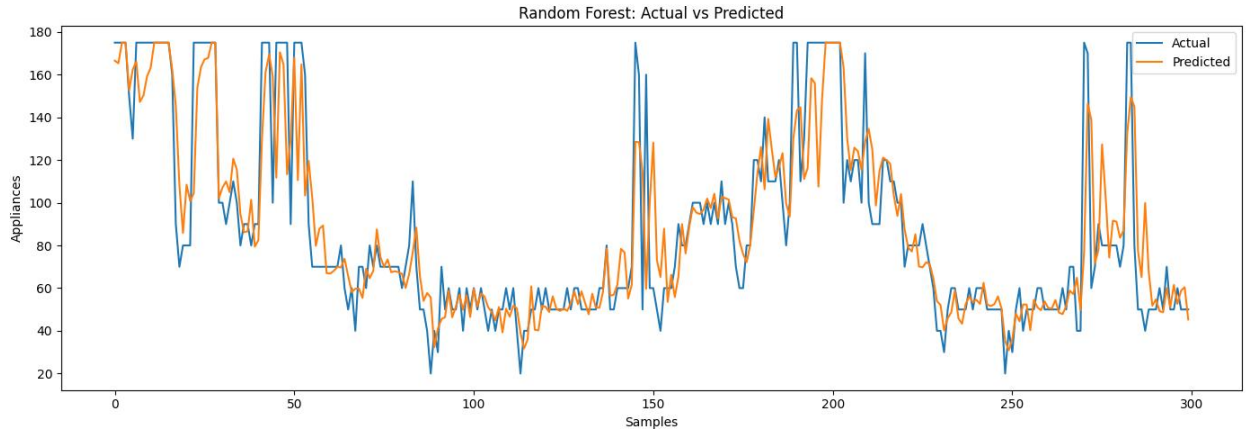

Figure 13: Random Forest Actual Vs Predicted

5.2 Deep Learning Model

5.2.1 LSTM (Long-Short Term Memory)

As a further exploration of sequence-based learning, an LSTM model was implemented. LSTM is a specific type of RNN (Recurrent Neural Network) introduced to capture temporal dependencies in sequential data. Unlike conventional machine learning models which treat each observation independently, LSTM focuses on the order and relationship between past observations.

In this assessment, LSTM was leveraged to model how historical energy consumption values and related attributes influence future energy consumption. This is especially suited for time-series forecasting, where historical data patterns usually play a critical role in determining future behavior.

The input features were transformed into sequences using a sliding window approach to prepare the data for the LSTM model. Each input instance consisted of a fixed number of previous time steps, enabling the model to learn temporal patterns more effectively over a defined time period.

5.2.2 Architecture

The architecture of the LSTM model consists of an LSTM layer followed by a dropout layer and fully connected dense layers. The LSTM layer focuses on learning temporal relationships in sequential data, while the dropout layer helps to minimize overfitting by randomly ignoring a fraction of neurons during training. The ReLU activation function was applied in the dense layer due to its computational efficiency and capability to minimize vanishing gradient problem. The model was trained with Adam optimizer, which is commonly relevant for deep learning tasks. And as the loss function for the optimization, Mean Squared Error (MSE was used.

```
model = Sequential(
    [
        LSTM(lstm_units, input_shape=input_shape),
        Dropout(dropout_rate),
        Dense(32, activation="relu"),
        Dense(1),
    ]
)

model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"],
)

return model
```

Figure 14: LSTM Architecture

To enhance generalization and avoid overfitting, early stopping was applied during training. This approach monitors validation loss and stops the training when no further improvement is observed, guaranteeing that the model retains the best-performing parameters without excessive training.

```
early_stopping = get_early_stopping_callback(patience=5)

history = lstm_model.fit(
    x_train_seq,
    y_train_seq,
    validation_split=0.1,
    epochs=30,
    batch_size=32,
    callbacks=[early_stopping],
    verbose=1,
)
```

Figure 15: LSTM Training Setup

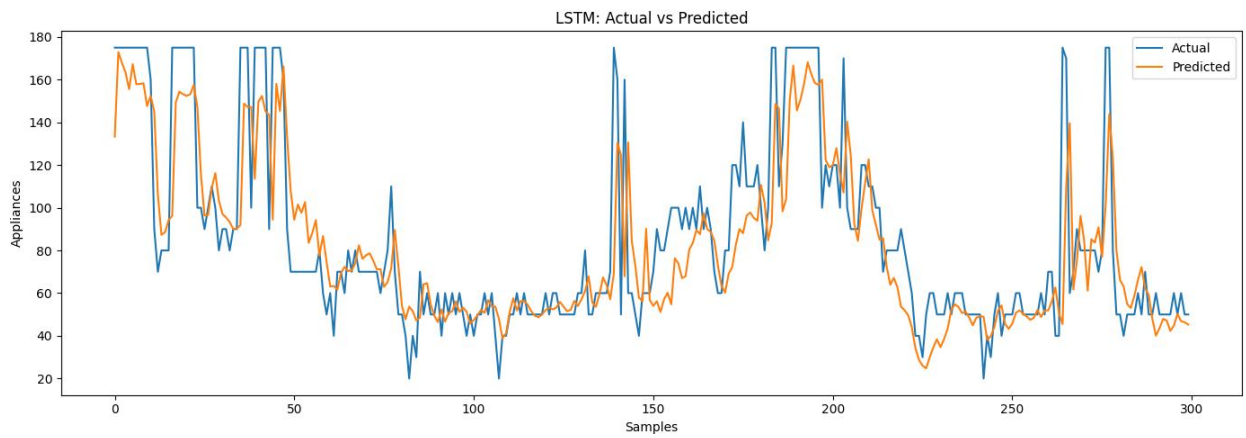

Figure 16: LSTM Actual Vs Predicted

5.3 Evaluation & Comparison

The performance of all the models was evaluated using the standard regression evaluation metrics of Mean Absolute Error (MAE), Root Mean Square Error (RMSE), Mean Absolute Percentage Error (MAPE), and R-Squared (R^2).

MAE measures the mean magnitude of prediction errors, while RMSE gives more weight to larger errors. MAPE expresses prediction error as a percentage, providing an informed understanding of model accuracy, and finally R^2 measures the proportion of variance in the target variable explained by the model.

Each model was trained on the training dataset and then evaluated against the test dataset to guarantee a fair comparison. The predictions generated by each model were compared against the actual values of appliance energy consumption. The results of the evaluation are summarized in Table 1.

Table 1: Model Comparison

Model	MAE	RMSE	MAPE (%)	R^2
Linear Regression	11.61	18.01	14.6	0.79
Random Forest	11.53	17.73	14.81	0.79
LSTM	14.72	22.82	18.33	0.66

Random Forest showed the best overall performance, by achieving the lowest MAE and RMSE values and the highest R^2 (shared with Linear Regression). This highlights that it was most effective in identifying the underlying patterns in the time-series data. Its ability to model complex feature interactions and non-linear relationships makes it especially suited for this dataset.

Linear Regression showed competitive results, with performance very closer to Random Forest. This indicates that the engineered features successfully grasped much of the underlying structure and temporal patterns in the data, that even a simpler linear model can perform well. However, as it achieved a slightly higher RMSE reflects that it is less effective in handling larger errors in relative to Random Forest approach.

The LSTM model showed results assigned with comparatively higher error value across all evaluation metrics and a lower R^2 score, showing weaker overall performance. Though LSTM is specifically designed to capture temporal dependencies, it struggled to accurately model extreme values and sharp fluctuations in the dataset. This may be due to reasons like insufficient training depth, limited sequence length, or the nature of the dataset, where well-engineered tabular attributes already provide strong predictive signals.

Overall, the results showcase that conventional machine learning models, especially Random Forest, can outperform even deep learning models such as LSTM when working with structured datasets consist with carefully engineered features. These insights highlight the importance of selecting models based on the characteristics of the dataset rather than relying solely on model complexity.

6. MODEL OPTIMIZATION

Following the initial model training and evaluation stage, model optimization stage was conducted to enhance forecasting performance and explore whether better results could be gained through parameter tuning and architectural adjustments. The optimization procedure focused on the two most relevant models, Random Forest which showed the best overall performance, and LSTM which has the potential to enhance with proper tuning due to its specialized ability to model sequential dependencies. The purpose of this stage was not only to reduce forecasting error but also to analyze how different setup and configuration might impact model behavior and generalization.

6.1 Random Forest Optimization

The Random Forest model optimization was done by tuning key hyperparameters which control the learning behavior and the complexity of the ensemble. For instance, the parameter of number of trees (*n_estimators*) was increased to enhance the model's ability to capture data trends across the dataset. A larger number of trees generally leads to more accurate and stable predictions, due to the benefit of averaging multiple decision paths.

Additionally, other parameters like maximum depth and minimum samples required for splitting were also adjusted to control the chance of overfitting. Limiting the depth of the trees avoids them from becoming overly complex and memorizing training data along with noise. Setting minimum sample thresholds makes sure that splits occur only when sufficient data is available.

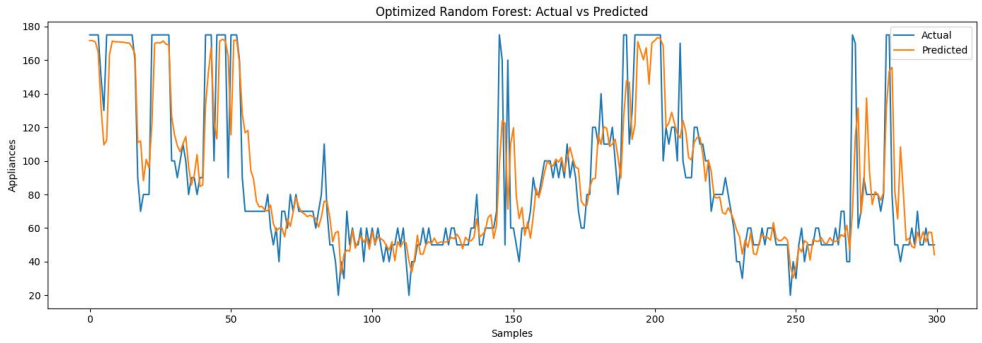

Figure 17: Optimized Random Forest Actual vs Predicted

Table 2: Random Forest Baseline vs Optimized

Model Version	MAE	RMSE	MAPE (%)	R^2
Baseline Random Forest	11.53	17.73	14.81	0.79
Optimized Random Forest	11.05	17.44	14.17	0.8

Through these parameter tuning, the Random Forest model showed a noticeable improvement in performance across all evaluation metrics. The optimized model achieved lower MAE (11.05) and RMSE (17.44) reflecting a reduction in both average prediction errors and the effect of larger errors. Also, the decrease in MAPE to 14.17% indicates improved relative prediction accuracy while the increase in R^2 suggests a better overall fit.

These advancements show that the optimized model has improved generalization capability and the ability to capture underlying patterns in the data. The Random Forest approach continues to effectively model non-linear relationships and complex feature interactions. And the optimizing process further refined its ability to handle variations and fluctuations in appliance energy consumption.

Overall, the optimization tuning process demonstrates the importance of hyperparameter tuning in enhancing models, even for already strong baseline models.

6.2 LSTM Optimization

The baseline LSTM model showcased reasonable performance but struggled to reduce prediction errors significantly. To address this, the architecture was improved by introducing a stacked design with two layers, consisting of 128 units in the 1st layer and 64 units in the following one. This allowed the LSTM model to learn more complex temporal dependencies. Dropout layers with a rate of 0.2 were added after each LSTM layer to minimize overfitting and enhance generalization. Adam optimizer was selected as the optimizer again since GD and SGD didn't improve the model performance as reflected through trial-and-error attempts. But the optimizer was used with a controlled learning rate of 0.0009 to ensure efficient and stable convergence during training. Early stopping with the patience of 10 was also included to prevent exhaustive overtraining.

Table 3: LSTM Baseline vs Optimized

Model Version	MAE	RMSE	MAPE	R^2
Baseline LSTM	14.72	22.82	18.33	0.66
Optimized LSTM	14.13	22.29	17.22	0.67

The optimized LSTM model was trained with initial epochs count as 35 with a batch size of 32, balancing stability and learning efficiency. Compared with the baseline LSTM, the optimized model achieved a very low improvement in performance in each metric just as Random Forest. Since the performance gain is very small, that emphasizes that additional data or further tuning may be required for significant performance improvement.

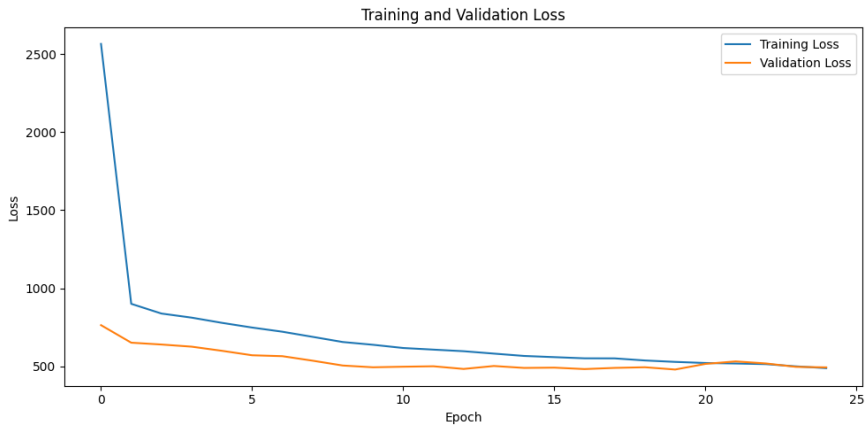

Figure 18: Baseline LSTM Training and Validation Loss

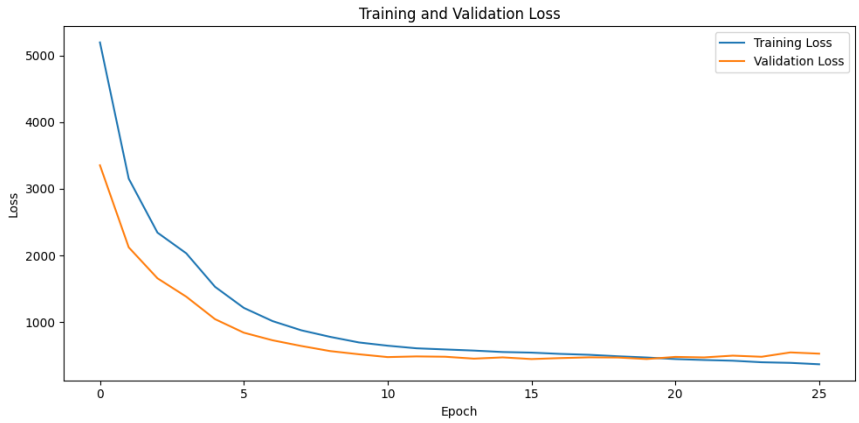

Figure 19: Optimized LSTM Training and Validation Loss

The training and validation loss curves for both baseline and optimized LSTM models demonstrate a consistent decrease over epochs, reflecting effective learning. The optimized model illustrates a smore smoother convergence and slightly lower validation loss compared to the baseline. This suggests that the generalization has improved. In spite of that, the overall improvement remains low, aligning with the marginal performance gains observed in evaluation metrics.

7. CHALLENGES AND SOLUTIONS

7.1 Data Preparation and Feature Engineering Challenges

One of the challenges occurred during feature engineering, especially when creating lag and rolling features. These engineering operations naturally introduced null values at the beginning of the dataset. They were removed during the end of feature engineering stage instead of applying imputation methods which might distort the nature of the time-series dataset. While removing, other considerations were raised such as whether this will affect the order of the dataset and whether this will significantly affect a considerable data loss. Since these missing values are introduced only in the beginning rows of the dataset removing them did not affect the time-series dataset order. And relative to the overall dataset size, these data removals were minimal.

Another challenge was handling outliers, particularly in the appliance energy suage values where sudden spikes were observed. These spikes along with the fluctuations reflect the real-world high consumption scenarios and are critical for accurate modeling. Removing them would have surely eliminated valuable information while disrupting the continuity of the time-series data. Therefore, an IQR-based clipping technique was applied to limit extreme values to certain boundaries while preserving the overall structure and variability of the dataset.

7.2 Model Selection and Performance Challenge

A considerable challenge involved in determining the most suitable model approach for the dataset after evaluation. Though deep learning models like LSTMs are particularly preferred for time-series data due to their ability to capture sequential dependencies, the model performance comparison showed that Random Forest model outperformed LSTM in this case. This revealed the importance of understanding the structure and the nature of the dataset as well-engineered features may reduce dependency on sequential learning.

Furthermore, the LSTM model faced challenges in capturing sharp fluctuations and extreme values in appliance energy usage. Even though it was trained on sequential data, it generated smoother predictions and struggled with high variability. This indicated that either additional tuning or more complex architecture would be required to leverage its full potential. Choosing these relevant modifications was a significant challenge as it was done through several trial-and-error attempts during model optimization phase.

8. CONCLUSION

8.1 Key Findings

This assessment explored several approaches for predicting appliance energy consumption leveraging a structured time-series dataset. After performing an EDA and relevant feature engineering procedures, it was observed that energy usage is strongly influenced by both environmental conditions and temporal patterns. The introduction of modified attributes such as time-based, lag, rolling and interaction features.

Out of the evaluated models, the Random Forest Regressor achieved the best overall performance, with the lowest MAE and RMSE values. This reflects that tree-based ensemble approaches are highly effective for structured datasets with well-engineered attributes. Linear Regression acted as a useful baseline through capturing general trends in a surprisingly good manner, but its performance was limited due to its assumption of linear relationships. The LSTM model did not outperform Random Forest although it is specifically designed for sequential data. This suggested that the engineered features have already captured a significant amount of the temporal information within the dataset.

Overall, the observations showcase that model performance does not solely depend on model complexity alone, but rather on how well the chosen model fits with the characteristics and nature of the dataset.

8.2 Limitations

Despite the presence of model optimization stage, the extent of hyperparameter tuning was relatively limited. More comprehensive tuning strategies like Bayesian optimization or grid search could have been applied to further enhance model performance. Similarly, the LSTM model could

benefit from further experimentation regarding deeper architectures, varied sequence lengths, or alternative configurations.

When considered from an implementation perspective, some limitations were identified in the overall code structure of the project. While utility functions were created to maintain code clarity and modularity, most of them could be further generalized to support broader use cases. Other minor inconsistencies in folder organization and project structure were observed which could be modified to improve clarity, maintainability, and scalability.

8.3 Future Enhancements

Based on the observations and results, further improvements can be considered for future work. One key improvement involves creating synthetic data for high appliance energy consumption ranges, especially for values above 120 because the Random Forest model showed struggle in accurately predicting these extreme values. Which might be caused by the lack of extreme appliance cases in training data. By enriching the dataset with more extreme representative samples, the model might learn these patterns and increase model performance.

Further improvements could involve experimenting with more advanced deep learning architectures like stacked LSTM models or hybrid models that combine both recurrent layers and convolution layers. Incorporating additional contextual features like occupancy information or external weather data may also enhance model performance.

Overall, these improvements would help to address the existing limitations and further strengthen the accuracy and robustness of the forecasting system.

9. REFERENCES

https://medium.com/@noorfatimaafzalbutt/outliers-detection-and-removal-using-iqr-method-e629aa8089a8

https://youtu.be/F8YeVV3DNKg?si=wpx299wjLdAfxFwh

https://youtu.be/b61DPVFX03I?si=WI_xpdDuwOoM3s09

10.APPENDIX

- GitHub Repository